EE604: Image Processing
Assignment 1

Student: Samruddhi Kedari
Roll No: 240535
Instructor: Koteswar Rao Jerripothula

Question 1]

Dataset Creation using the MNIST Dataset

a part

Obtain foreground segmentation masks for the images in the MNIST dataset using Otsu's thresholding

Solution

The python code solution is as follows:

```
import numpy as np  #Importing required libraries and mnist dataset
import cv2
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
import os


(imgtrain, labeltrain), (imgtest, labeltest) = mnist.load_data() #Loading training and testing labels and images


print("Input MNIST training data shape:", imgtrain.shape)  #Printing this message for verification that dataset is correct
print("Input MNIST testing data shape:", imgtest.shape)


def getforegroundmasks(images): #Defining a function for ostu's thresholding
    masks = [] #making an empty array for storing mask
    for img in images:
        img = img.astype(np.uint8) #ensuring that each pixel is 8 bit


        threshold, mast = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) #applying ostu's thresholding

        #Slight enhancement in results using morphological operations
        #kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (2, 2))
        #mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)




        masks.append(mask) #putting all masks in array created earlier
    return np.array(masks)


trainmasks = getforegroundmasks(imgtrain) #getting masks for training images
testmasks = getforegroundmasks(imgtest) #getting masks for testing images


print("Foreground mask training data shape:", trainmasks.shape) #Printing size of masks created which wil be similar to input dataset
print("Foreground mask testing data shape:", testmasks.shape)




n = 6 #printing some results
plt.figure(figsize=(10, 4))
```

```python
for i in range(n):
    plt.subplot(2, n, i+1)
    plt.imshow(imgtrain[i], cmap='gray')  #Plotting MNIST Images
    plt.title("Input MNIST")
    plt.axis('off')


    plt.subplot(2, n, i+1+n)
    plt.imshow(trainmasks[i], cmap='gray') #Plotting corresponding masks created
    plt.title("Foreground mask")
    plt.axis('off')


plt.tight_layout() #Displaying results
plt.show()


os.makedirs("mnistforegrounddataset", exist_ok=True)  #Saving mask dataset created for further use
np.save("mnistforegrounddataset/trainmasks.npy", trainmasks)
np.save("mnistforegrounddataset/testmasks.npy", testmasks)
```

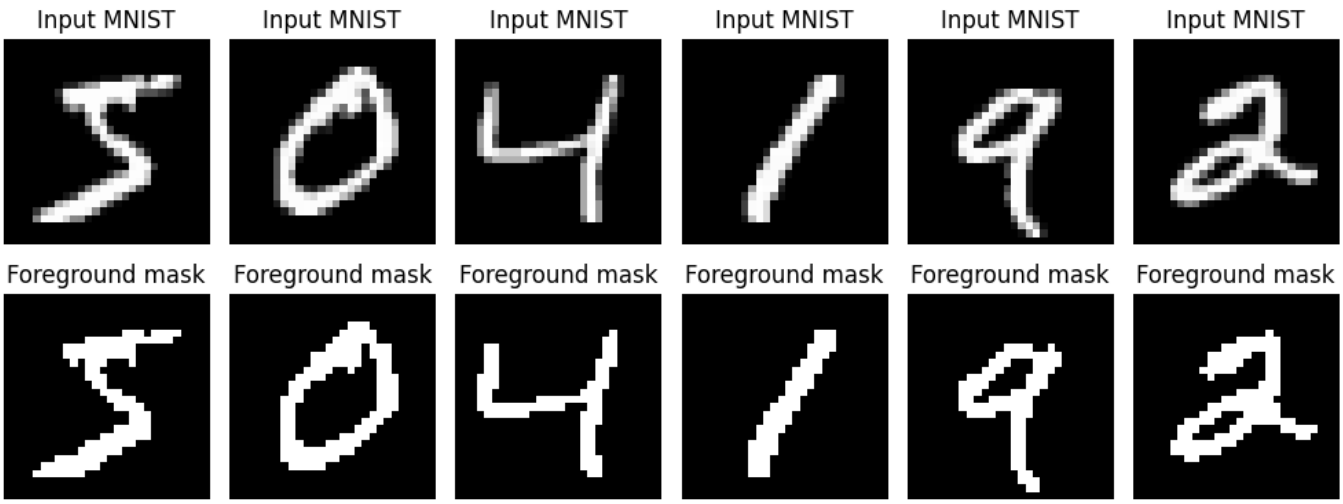

Figure 1: Sample output results

0.1 Logic Explanation

0.1.1 Otsu's Thresholding

- Otsu's method automatically determines the optimal threshold value to separate the foreground (digit) from the background (black area).
- It minimizes the **intra-class variance**, which means it reduces the difference within the foreground and background pixel intensities.
- The thresholding rule is applied as follows:

$$\text{If pixel intensity} > \text{threshold} \Rightarrow \text{foreground (white} = 255)$$

$$\text{Else} \Rightarrow \text{background (black} = 0)$$

- Hence, we obtain a binary mask of the same size (28×28).

0.2 Enhancements

0.2.1 Usage of morphological operation for better results

```python

kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (2, 2))
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

- Filling small holes inside the foreground (digit) to ensure solid and continuous white regions.
- Connecting small gaps between broken white regions to improve the completeness of the digit shape.
- Smoothing the outer boundary of the digit to achieve a cleaner and more defined contour.
- Reducing noise in the binary mask by removing isolated background or foreground pixels.

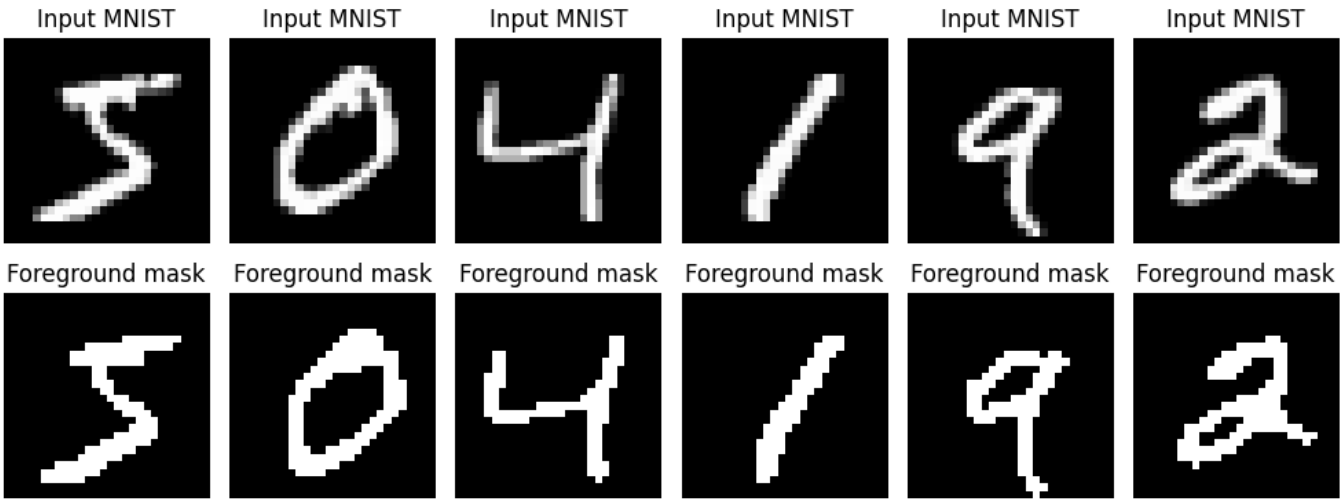

Figure 2: Sample output results with morphological operations

b part

Generate a tight enclosing circle around the mask.

Solution

The python code solution is as follows:

```
import numpy as np  #Importing required libraries and mnist dataset
import cv2
from tensorflow.keras.datasets import mnist
import matplotlib.pyplot as plt
import os


(imgtrain, labeltrain), (imgtest, labeltest) = mnist.load_data() #Loading training and testing
    labels and images


print("Input MNIST training data shape:", imgtrain.shape)  #Printing this message for verification
    that dataset is correct
print("Input MNIST testing data shape:", imgtest.shape)


def getforegroundmasks(images): #Defining a function for ostu's thresholding
    masks = [] #making an empty array for storing mask
    for img in images:
        img = img.astype(np.uint8) #ensuring that each pixel is 8 bit


        threshold, mask = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU) #applying
            ostu's thresholding




        masks.append(mask) #putting all masks in array created earlier
    return np.array(masks)


trainmasks = getforegroundmasks(imgtrain) #getting masks for training images
```

```python
testmasks = getforegroundmasks(imgtest) #getting masks for testing images


print("Foreground mask training data shape:", trainmasks.shape) #Printing size of masks created which wil be similar to input dataset
print("Foreground mask testing data shape:", testmasks.shape)


def generatecirclemasks(masks): #Defining function to generate tight enclosing circle
    circlemasks = [] #creating an array to store circular masks
    for mask in masks:
        mask = mask.astype(np.uint8) #ensuring that each mask is of 8 bits
        contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE) #finding boarders of image


        if len(contours) > 0: #If such contours found then
            actcontour = max(contours, key=cv2.contourArea) #getting actual contour of digit from maximum of all contours
            (xcent, ycent), radius = cv2.minEnclosingCircle(actcontour) #computing parameters of minimum enclosing circle of actcontour
            center = (int(xcent), int(ycent)) #getting int value from float
            radius = int(radius) #getting int value of radius
        else:
            center, radius = (14, 14), 0 #In case no contour found then use these parameters


        circlemask = np.zeros_like(mask) #Creating blank mask for circle
        cv2.circle(circlemask, center, radius, 255, -1)  #Making white circle in the mask


        '''
        #enhancing circular masks with morphological operations
        kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (3, 3)) #Stucturing element
        circlemask = cv2.dilate(circlemask, kernel, iterations=1) #Using dilation function to smooth circle contour
        '''
        circlemasks.append(circlemask) #Storing mask in the array of circlemasks


    return np.array(circlemasks) #returing array


traincirclemasks = generatecirclemasks(trainmasks) #generating circular masks for training masks
testcirclemasks = generatecirclemasks(testmasks) #generating circular masks for testing masks


print("Train circle masks shape:", traincirclemasks.shape) #Getting size of circular masks which will similar to input masks


n = 5 #Some example results
plt.figure(figsize=(10, 6))
for i in range(n):
    plt.subplot(3, n, i+1)
    plt.imshow(imgtrain[i], cmap='gray')
    plt.title(f"Input MNIST ({labeltrain[i]})") #Plotting input mnist images with thier classes
    plt.axis('off')


    plt.subplot(3, n, i+1+n)
    plt.imshow(trainmasks[i], cmap='gray')
    plt.title("Foreground Mask")#Plttoing foreground masks created
    plt.axis('off')


    plt.subplot(3, n, i+1+2*n)
    plt.imshow(traincirclemasks[i], cmap='gray')
    plt.title("Circle Mask") #Plotting corresponding circular masks created
    plt.axis('off')


plt.tight_layout()
plt.show() #Displaying results


os.makedirs("mnistcirclizeddataset", exist_ok=True)
np.save("mnistcirclizeddataset/trainimages.npy", imgtrain)
np.save("mnistcirclizeddataset/traincirclemasks.npy", traincirclemasks)
```

```
np.save("mnistcirclizeddataset/trainlabels.npy", labeltrain)


np.save("mnistcirclizeddataset/testimages.npy", imgtest)
np.save("mnistcirclizeddataset/testcirclemasks.npy", testcirclemasks)
np.save("mnistcirclizeddataset/testlabels.npy", labeltest)


print("Unique labels in training data:", np.unique(labeltrain)) #Printing classes present
```

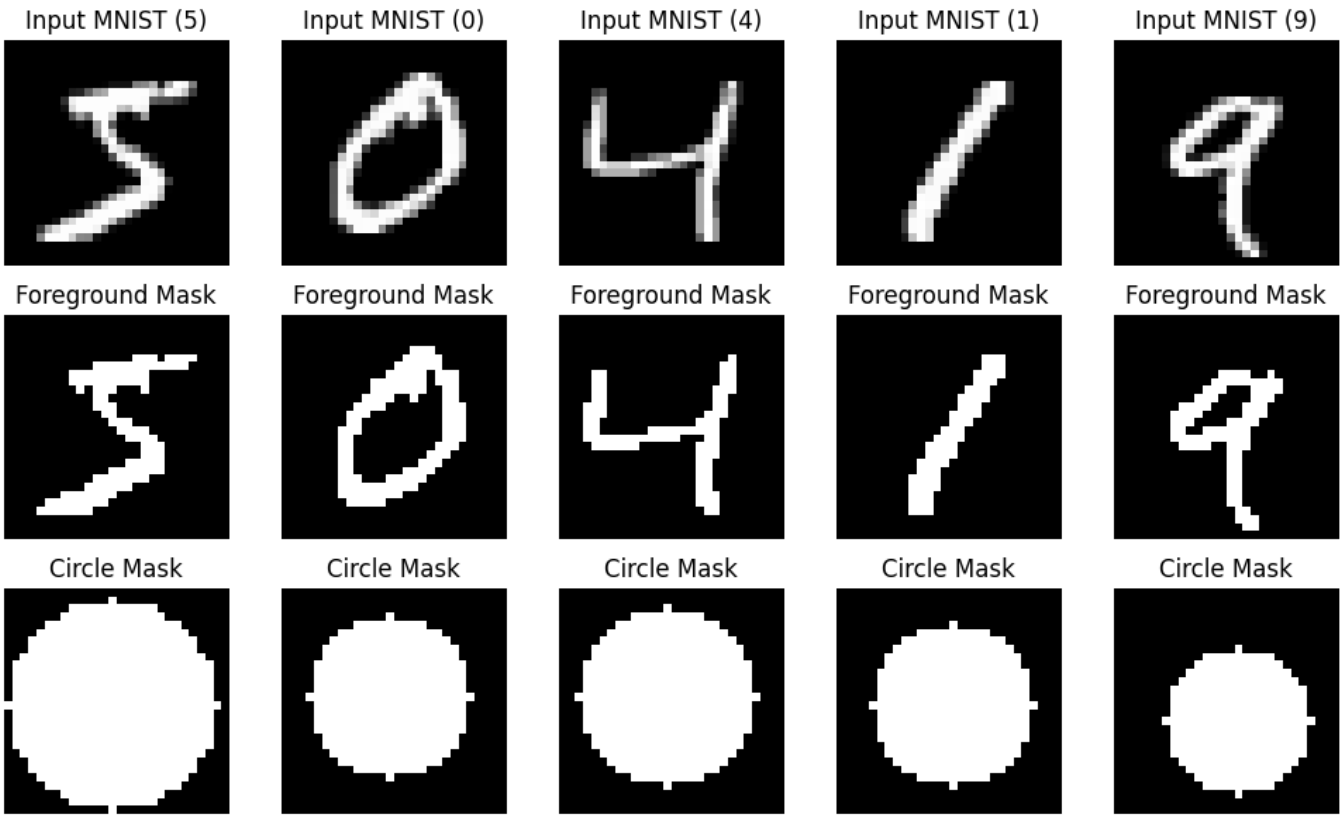

Figure 3: Tight enclosing circles generated

0.3 Logic Explanation

- Identify the white digit region in each binary mask to locate the foreground area.
- Detect the outer boundary (contour) of the digit using contour detection techniques.
- Select the largest contour as the main digit shape to ensure the correct region is captured.
- Compute the smallest enclosing circle that fully covers this contour.
- Draw a filled white circle on a blank mask using the obtained center and radius.
- If no valid contour is detected, generate a default small circular region as a fallback.
- Store all the resulting circular masks to form the final processed dataset.

0.4 Enhancements

Same like part a use of morphological operations can be done

c part

Concatenate four images and their corresponding ground truths in a 2×2 grid

Solution

The python code solution is as follows:

```python
import numpy as np    #Importing required libraries
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
import os


(imgtrain, _), (imgtest, _) = mnist.load_data() #Getting test and training images
images = np.concatenate([imgtrain, imgtest], axis=0)  # total 70,000 images after merging training and testing images


trainmasks = np.load("/content/mnistforegrounddataset/trainmasks.npy") #Path to foreground masks created earlier
testmasks  = np.load("/content/mnistforegrounddataset/testmasks.npy")
masks = np.concatenate([trainmasks, testmasks], axis=0) #Similarly merging both test and training masks to get 70000 images


print("Images shape:", images.shape) #Getting size of images and masks
print("Masks shape:", masks.shape)


def creategridimage(imgs, masks): #Function to create 2 cross 2 grid

    top = np.hstack((imgs[0], imgs[1]))    #Top row of the grid
    bottom = np.hstack((imgs[2], imgs[3])) # bottom row of the grid
    gridimage = np.vstack((top, bottom)) #Combining top and bottom rows to get grid of images


    topm = np.hstack((masks[0], masks[1])) #similarly getting grid of masks
    bottomm = np.hstack((masks[2], masks[3]))
    gridmask = np.vstack((topm, bottomm))


    gridmask = (gridmask > 0).astype(np.uint8) #Ensuring that mask grid is binary


    return gridimage, gridmask


numsamples = 250000   #Ensuring that there are total 250000 images in the dataset
combinedimages = [] #Creating array for combined images and masks
combinedmasks = []


for _ in range(numsamples):
    index = np.random.choice(len(images), 4, replace=False) #Getting 4 different and random indexes of array of images
    imgs = [images[i] for i in index] #Creating array of images from those 4 values of indexes
    msks = [masks[i] for i in index]


    imggrid, maskgrid = creategridimage(imgs, msks) #Getting grid of images and masks
    combinedimages.append(imggrid) #Storing results in an array made earlier
    combinedmasks.append(maskgrid)


combinedimages = np.array(combinedimages, dtype=np.uint8) #converting to numpy arrays for storage
combinedmasks = np.array(combinedmasks, dtype=np.uint8)


print("\nGenerated dataset shapes:")  #Printing sizes of images ,  masks and grids
print("Images:", combinedimages.shape)
print("Masks:", combinedmasks.shape)
print("Each grid image shape:", combinedimages[0].shape)
print("Each grid mask shape:", combinedmasks[0].shape)
print("Unique values in masks:", np.unique(combinedmasks))


os.makedirs("mnistsemanticsegdataset", exist_ok=True) #Saving dataset
np.save("mnistsemanticsegdataset/gridimages.npy", combinedimages)
np.save("mnistsemanticsegdataset/gridmasks.npy", combinedmasks)


#Plottimg sample results
```

```python
n = 3
plt.figure(figsize=(10, 6))
for i in range(n):
    plt.subplot(2, n, i + 1)
    plt.imshow(combinedimages[i], cmap='gray')
    plt.title("Grid of Image")
    plt.axis('off')


    plt.subplot(2, n, i + 1 + n)
    plt.imshow(combinedmasks[i], cmap='gray')
    plt.title("Grid of Mask")
    plt.axis('off')


plt.tight_layout() #Plotting results
plt.show()


totalimgs = combinedimages.shape[0] #Getting total number of images in the dataset
totalmasks = combinedmasks.shape[0]
uniquevals = np.unique(combinedmasks)
isbinary = np.array_equal(uniquevals, [0, 1]) #ensuring that masks are binary


print(f"\n Total combined images: {totalimgs}")
print(f" Total combined masks: {totalmasks}")
print(f" Unique values in masks: {uniquevals}")
print(f" Masks are binary: {isbinary}")
```

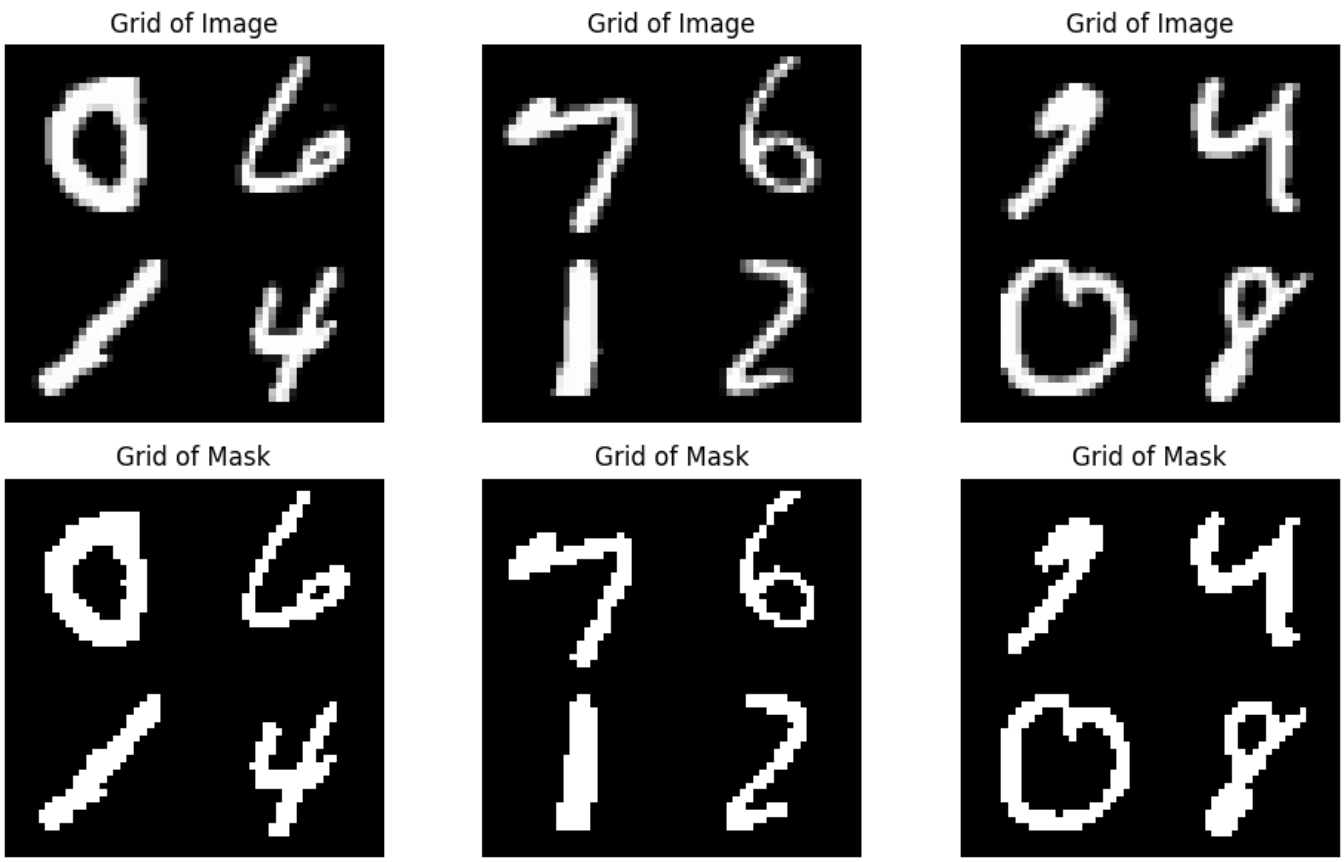

Figure 4: Sample results of grid generation

0.5 Logic Explanation

- Define the `creategridimage()` function to generate composite images and their corresponding masks.
- Randomly select four images along with their four binary masks from the dataset.
- Combine the selected images into a 2×2 grid:

  - Top row – place two images side by side using `np.hstack()`.
  - Bottom row – place the next two images side by side.
  - Stack both rows vertically using `np.vstack()` to form the complete grid.

- Perform the same operation for the corresponding masks to form a binary grid mask.
- Repeat this grid creation process 250,000 times using random selections, ensuring no duplicate images within a single grid.
- Store all combined grid images and their corresponding masks as NumPy arrays for dataset generation.

Question 2]

Train a deep learning network from scratch to perform foreground extraction on the dataset created in Q1(a).

Solution

The python code solution is as follows:

```
import numpy as np                    #Importing required libraries and datasets
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist


(imgtrain, _), (imgtest, _) = mnist.load_data() #Getting testing and training iamges from mnist dataset


imgtrain = imgtrain.astype("float32") / 255.0 #Normalising them to floating number betweem 0 and 1
imgtest  = imgtest.astype("float32") / 255.0  #Its done so that CNN can be trained easily


imgtrain = np.expand_dims(imgtrain, -1) #Adding channel dimension as expected by CNN
imgtest  = np.expand_dims(imgtest, -1)


preimgtrain = np.load("/content/mnistforegrounddataset/trainmasks.npy") #Laoding previoulsy saved foreground masks
preimgtest  = np.load("/content/mnistforegrounddataset/testmasks.npy")


preimgtrain  = np.expand_dims((preimgtrain  > 0).astype("float32"), -1) #ensuring that images are bianry and converting them to float value
preimgtest  = np.expand_dims((preimgtest > 0).astype("float32"), -1)


print("imgtrain shape:",imgtrain .shape)      #for confirming input and output sizes
print("preimgtrain  shape:", preimgtrain .shape)
print("imgtest shape:",imgtest .shape)
print("preimgtest shape:", preimgtest.shape)


def buildunet(inputshape=(28, 28, 1)): #defining a function to build a model
    inputs = layers.Input(shape=inputshape) #Defining input layer of our Neural network
    #Encoding
    convo1 = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs) #First convolutional layer
    pool1 = layers.MaxPooling2D()(convo1) #Pooling is done to reduce spatial size so that only imp features remains


    convo2 = layers.Conv2D(64, 3, activation='relu', padding='same')(pool1) #another convolutional layer
    pool2 = layers.MaxPooling2D()(convo2) #another pooling layer


    bot1 = layers.Conv2D(128, 3, activation='relu', padding='same')(pool2) #For getting high level features
    #decoding
```

```python
    upsamp1 = layers.UpSampling2D()(bot1) #upsampling feature map from bottleneck
    concat1 = layers.Concatenate()([upsamp1, convo2]) #merging upsampled features with encoded features
    convo3 = layers.Conv2D(64, 3, activation='relu', padding='same')(concat1) #Refining merged feature map with convolutional layer


    upsamp2 = layers.UpSampling2D()(convo3) #Upsampling output of last step
    concat2 = layers.Concatenate()([upsamp2, convo1]) #merging done
    convo4 = layers.Conv2D(32, 3, activation='relu', padding='same')(concat2) #Refining done using convolutional layer


    outputs = layers.Conv2D(1, 1, activation='sigmoid')(convo4) #single channel output


    model = models.Model(inputs, outputs) #Wrapping all the layers in model object
    return model


model = buildunet() #printing layer by layer summary
model.summary()


model.compile(optimizer='adam',                #Defining metric , optimizer and loss
              loss='binary_crossentropy',
              metrics=['accuracy'])


history = model.fit(
   imgtrain , preimgtrain ,     #Setting cycles as 5 and batchsize as 100 so that overall accuracy is more and code is not too slow
    validation_data=(imgtest, preimgtest),
    epochs=5,
    batch_size=100
)


def IOUmetric(trueout, predout, threshold=0.5): #Function for getting IOU metrice
    predbinout = (predout> threshold).astype(np.float32) #Converting to binary
    intersection = np.sum(trueout * predbinout) #Total number of correct foreground pixels
    union = np.sum(trueout) + np.sum(predbinout) - intersection #getting union area of both predicted and true outputs
    return intersection / union if union != 0 else 0 #Getting IOU which is intersetion \ union


pred = model.predict(imgtest) #Getting models predicted masks on the input data
iouscore = IOUmetric(preimgtest, pred)
print(f"IoU Score: {iouscore:.4f}")


n = 5 #Plotting sample results
plt.figure(figsize=(10, 4))
for i in range(n):       #Input images
    plt.subplot(3, n, i + 1)
    plt.imshow(imgtest[i].squeeze(), cmap='gray')
    plt.title("Input Image")
    plt.axis('off')


    plt.subplot(3, n, i + 1 + n) #Foreground masks which were already saved
    plt.imshow(preimgtest[i].squeeze(), cmap='gray')
    plt.title("Ground Truth")
    plt.axis('off')


    plt.subplot(3, n, i + 1 + 2*n) #Predicted mask by training model
    plt.imshow(pred[i].squeeze(), cmap='gray')
    plt.title("Predicted Mask")
    plt.axis('off')


plt.tight_layout() #Displaying output
plt.show()
```

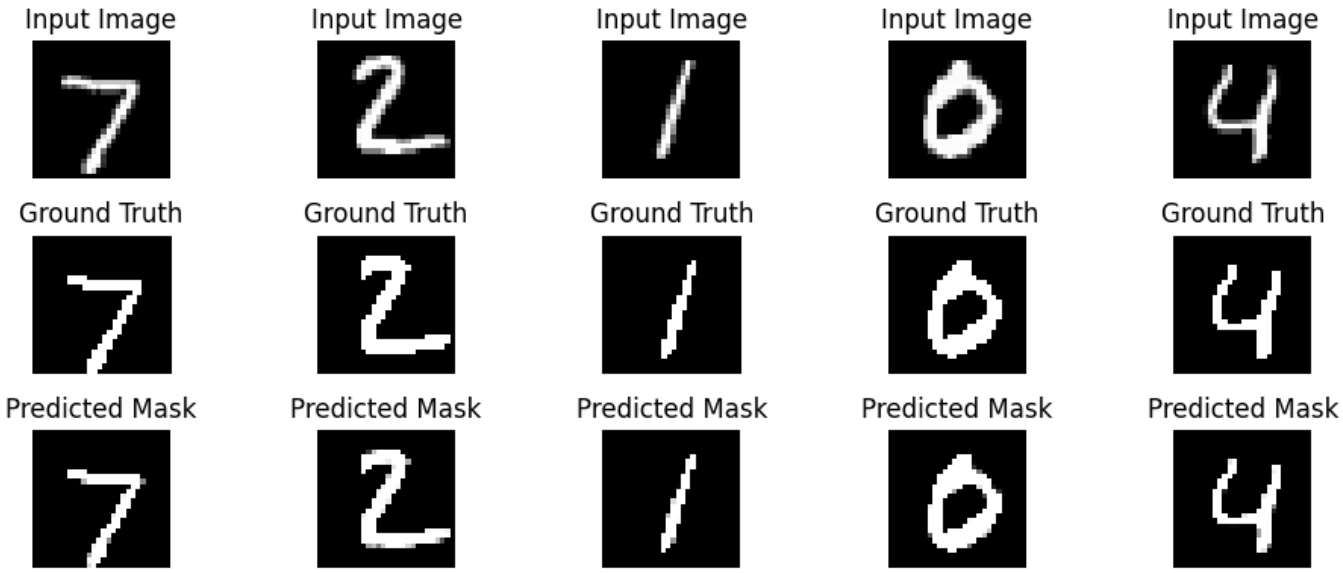

Figure 5: Sample output results

```
 Total params: 231,041 (902.50 KB)
 Trainable params: 231,041 (902.50 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 537s 890ms/step - accuracy: 0.9643 - loss: 0.0883 - val_accuracy: 0.9926 - val_loss: 0.0174
Epoch 2/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 533s 842ms/step - accuracy: 0.9932 - loss: 0.0162 - val_accuracy: 0.9946 - val_loss: 0.0132
Epoch 3/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 582s 875ms/step - accuracy: 0.9948 - loss: 0.0126 - val_accuracy: 0.9957 - val_loss: 0.0107
Epoch 4/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 502s 837ms/step - accuracy: 0.9959 - loss: 0.0101 - val_accuracy: 0.9966 - val_loss: 0.0085
Epoch 5/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 582s 871ms/step - accuracy: 0.9968 - loss: 0.0080 - val_accuracy: 0.9969 - val_loss: 0.0076
313/313 ━━━━━━━━━━━━━━━━━━━━ 23s 72ms/step
IoU Score: 0.9782
```

Figure 6: IoU Calculation

0.6 IoU Score = 0.9782

$$IoU = \frac{|X \cap Y|}{|X \cup Y|}$$

where X represents the set of pixels in the predicted mask, and Y represents the set of pixels in the ground truth mask. The numerator $|X \cap Y|$ denotes the intersection (common pixels), while the denominator $|X \cup Y|$ denotes the union.

0.7 Logic Explanation

U-Net Based CNN for Foreground Segmentation
The U-Net based Convolutional Neural Network (CNN) is designed for the foreground segmentation of MNIST digit images.

Dataset Preparation

- The MNIST dataset is loaded using `tensorflow.keras.datasets`.
- All input images are normalized to the range $[0, 1]$ to ensure faster and more stable CNN training.
- Corresponding foreground masks, previously generated using Otsu's thresholding method, are loaded and converted into binary form, where 0 represents the background and 1 represents the foreground.

Model Architecture – U-Net

The U-Net architecture follows a symmetric **encoder–decoder** design that enables precise pixel-level segmentation.

- **Encoder (Contracting Path):** Consists of two convolutional layers (with 32 and 64 filters) followed by MaxPooling layers. This path extracts spatial and texture features while progressively reducing image dimensions.
- **Bottleneck Layer:** Contains a deeper convolutional block (128 filters) that captures high-level semantic representations of the input image.
- **Decoder (Expanding Path):** Uses UpSampling layers to gradually restore spatial resolution. Skip connections concatenate encoder features with decoder outputs, preserving fine boundary details lost during pooling.
- **Output Layer:** A final 1×1 convolution layer with a sigmoid activation function generates a single-channel mask that represents the pixel-wise probability of the foreground region.

Training Setup

- The model is compiled using the `Adam` optimizer with binary cross-entropy loss and accuracy as the evaluation metric.
- Training is performed for 5 epochs with a batch size of 100, achieving a balance between training speed and segmentation accuracy.

Evaluation

- The trained model's predictions are quantitatively evaluated using the **Intersection-over-Union (IoU)** metric to measure segmentation performance.
- Sample inputs, ground-truth masks, and predicted masks are visualized to assess qualitative performance.

Question 3]

Train a deep learning network from scratch to perform classification with circlization on the dataset created in Q1(b).

Solution

The python code solution is as follows:

```python
import numpy as np                    #Importing required libraries and datasets
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist


(imgtrain, _), (imgtest, _) = mnist.load_data() #Getting testing and training iamges from mnist dataset


imgtrain = imgtrain.astype("float32") / 255.0 #Normalising them to floating number betweem 0 and 1
imgtest  = imgtest.astype("float32") / 255.0  #Its done so that CNN can be trained easily


imgtrain = np.expand_dims(imgtrain, -1) #Adding channel dimension as expected by CNN
imgtest  = np.expand_dims(imgtest, -1)


preimgtrain = np.load("/content/mnistforegrounddataset/trainmasks.npy") #Laoding previoulsy saved foreground masks
```

```python
preimgtest  = np.load("/content/mnistforegrounddataset/testmasks.npy")


preimgtrain  = np.expand_dims((preimgtrain  > 0).astype("float32"), -1) #ensuring that images are biarry and converting them to float value
preimgtest  = np.expand_dims((preimgtest > 0).astype("float32"), -1)


print("imgtrain shape:",imgtrain .shape)      #for confirming input and output sizes
print("preimgtrain  shape:", preimgtrain .shape)
print("imgtest shape:",imgtest .shape)
print("preimgtest shape:", preimgtest.shape)


def buildunet(inputshape=(28, 28, 1)): #defining a function to build a model
    inputs = layers.Input(shape=inputshape) #Defining input layer of our Neural network
    #Encoding
    convo1 = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs) #First convolutional layer
    pool1 = layers.MaxPooling2D()(convo1) #Pooling is done to reduce spatial size so that only imp features remains


    convo2 = layers.Conv2D(64, 3, activation='relu', padding='same')(pool1) #another convolutional layer
    pool2 = layers.MaxPooling2D()(convo2) #another pooling layer


    bot1 = layers.Conv2D(128, 3, activation='relu', padding='same')(pool2) #For getting high level features
    #decoding
    upsamp1 = layers.UpSampling2D()(bot1) #upsampling feature map from bottleneck
    concat1 = layers.Concatenate()([upsamp1, convo2]) #merging upsampled features with encoded features
    convo3 = layers.Conv2D(64, 3, activation='relu', padding='same')(concat1) #Refining merged feature map with convolutional layer


    upsamp2 = layers.UpSampling2D()(convo3) #Upsampling output of last step
    concat2 = layers.Concatenate()([upsamp2, convo1]) #merging done
    convo4 = layers.Conv2D(32, 3, activation='relu', padding='same')(concat2) #Refining done using convolutional layer


    outputs = layers.Conv2D(1, 1, activation='sigmoid')(convo4) #single channel output


    model = models.Model(inputs, outputs) #Wrapping all the layers in model object
    return model


model = buildunet() #printing layer by layer summary
model.summary()


model.compile(optimizer='adam',                #Defining metric , optimizer and loss
              loss='binary_crossentropy',
              metrics=['accuracy'])


history = model.fit(
   imgtrain , preimgtrain ,     #Setting cycles as 5 and batchsize as 100 so that overall accuracy is more and code is not too slow
    validation_data=(imgtest, preimgtest),
    epochs=5,
    batch_size=100
)


def IOUmetric(trueout, predout, threshold=0.5): #Function for getting IOU metrice
    predbinout = (predout> threshold).astype(np.float32) #Converting to binary
    intersection = np.sum(trueout * predbinout) #Total number of correct foreground pixels
    union = np.sum(trueout) + np.sum(predbinout) - intersection #getting union area of both predicted and true outputs
    return intersection / union if union != 0 else 0 #Getting IOU which is intersetion \ union


pred = model.predict(imgtest) #Getting models predicted masks on the input data
iouscore = IOUmetric(preimgtest, pred)
```

```python
print(f"IoU Score: {iouscore:.4f}")


n = 5 #Plotting sample results
plt.figure(figsize=(10, 4))
for i in range(n):        #Input images
    plt.subplot(3, n, i + 1)
    plt.imshow(imgtest[i].squeeze(), cmap='gray')
    plt.title("Input Image")
    plt.axis('off')


    plt.subplot(3, n, i + 1 + n) #Foreground masks which were already saved
    plt.imshow(preimgtest[i].squeeze(), cmap='gray')
    plt.title("Ground Truth")
    plt.axis('off')


    plt.subplot(3, n, i + 1 + 2*n) #Predicted mask by training model
    plt.imshow(pred[i].squeeze(), cmap='gray')
    plt.title("Predicted Mask")
    plt.axis('off')


plt.tight_layout() #Displaying output
plt.show()
```

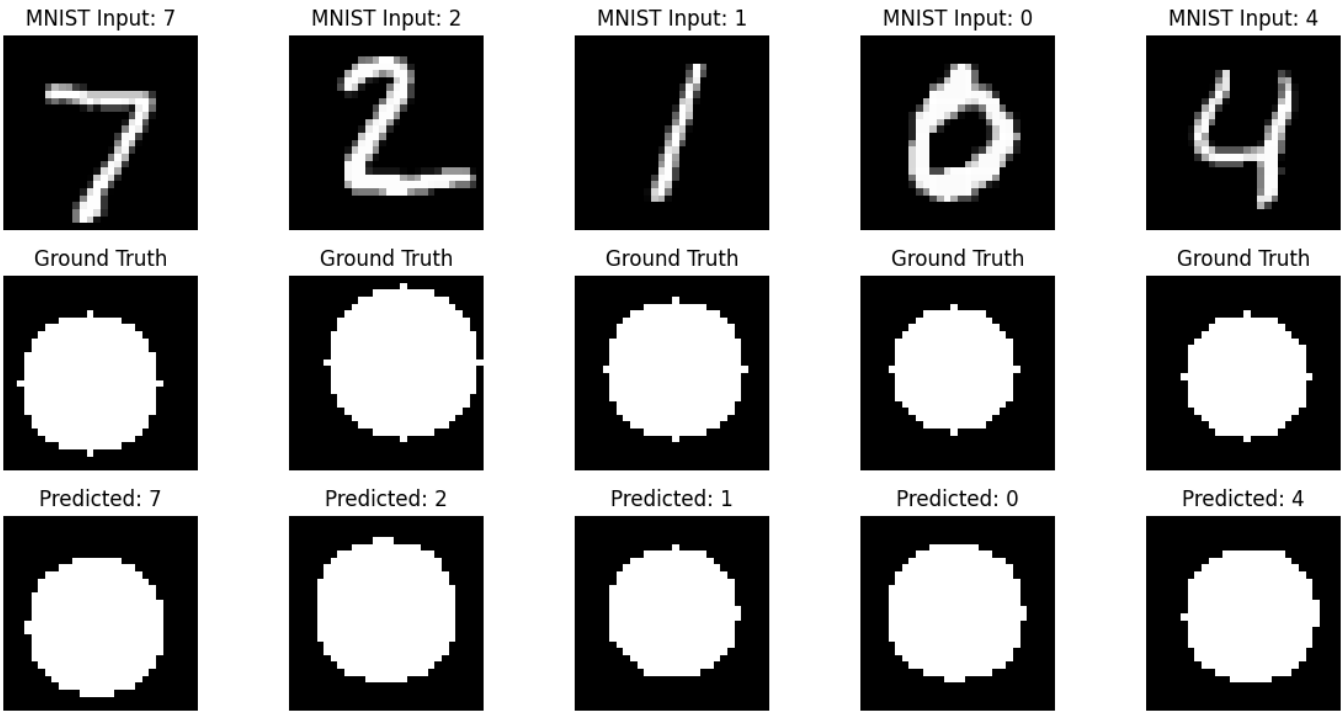

Figure 7: Sample output results

```
 Total params: 101,578 (396.79 KB)
 Trainable params: 101,578 (396.79 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 93s 151ms/step - accuracy: 0.6060 - loss: 1.1556 - val_accuracy: 0.9316 - val_loss: 0.2249
Epoch 2/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 90s 150ms/step - accuracy: 0.9310 - loss: 0.2284 - val_accuracy: 0.9554 - val_loss: 0.1415
Epoch 3/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 142s 150ms/step - accuracy: 0.9558 - loss: 0.1466 - val_accuracy: 0.9621 - val_loss: 0.1193
Epoch 4/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 142s 150ms/step - accuracy: 0.9634 - loss: 0.1157 - val_accuracy: 0.9726 - val_loss: 0.0872
Epoch 5/5
600/600 ━━━━━━━━━━━━━━━━━━━━ 144s 153ms/step - accuracy: 0.9702 - loss: 0.0969 - val_accuracy: 0.9774 - val_loss: 0.0681
313/313 ━━━━━━━━━━━━━━━━━━━━ 6s 18ms/step

Mean IoU Score: 0.8070
```

Figure 8: IoU Calculation

0.8 IoU Score: 0.8070

$$IoU = \frac{|X \cap Y|}{|X \cup Y|}$$

where X represents the set of pixels in the predicted mask, and Y represents the set of pixels in the ground truth mask. The numerator $|X \cap Y|$ denotes the intersection (common pixels), while the denominator $|X \cup Y|$ denotes the union.

0.9 Logic Explanation

CNN-Based Digit Classification and Circular Mask Prediction
The Convolutional Neural Network (CNN) model is designed to classify MNIST digits and associate each class with a representative circular mask.

Dataset Preparation

- The MNIST dataset is loaded using `tensorflow.keras.datasets`.
- Input images are normalized to the range $[0, 1]$ to stabilize and speed up CNN training.
- Each grayscale image is expanded to have a single channel ($28 \times 28 \times 1$), suitable for CNN input.
- Corresponding circular ground truth masks are loaded, binarized, and reshaped to match image dimensions.

Model Architecture – CNN

The CNN architecture follows a hierarchical feature extraction process for digit classification.

- **Convolutional Layers:** Three convolutional layers with 32, 64, and 128 filters extract low to high-level spatial features using ReLU activation and *same* padding.
- **Pooling Layers:** MaxPooling layers follow each convolution to down-sample feature maps and reduce spatial dimensions.
- **Global Pooling:** A Global Average Pooling layer condenses spatial information into a compact representation.
- **Fully Connected Layers:** A dense layer with 64 neurons learns abstract features, followed by a softmax output layer that classifies digits into 10 categories.

Training Setup

- The model is compiled with the `Adam` optimizer and the sparse categorical cross-entropy loss function.
- Training is performed for 5 epochs with a batch size of 100 for optimal accuracy and efficiency.

Evaluation

- Class-specific mean circular masks are generated to represent predicted digit shapes.
- The **Intersection-over-Union (IoU)** metric evaluates overlap between predicted and ground truth masks.
- Visualization of test samples, ground-truth circles, and predicted masks is performed for qualitative analysis.

Question 4]

Train a deep learning network from scratch to perform semantic segmentation on the dataset created in Q1(c).

Solution

The python code solution is as follows:

```
import numpy as np #Importing required libraries
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models, backend as K
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split

images = np.load("/content/mnistsemanticsegdataset/gridimages.npy") #Loading datasets
masks = np.load("/content/mnistsemanticsegdataset/gridmasks.npy")

print("Original dataset size:", images.shape) #Printing size of original ground dataset

images = images[:20000] #Only small portion of data is used as otherwise it is crashing my system
masks = masks[:20000]

print("Using subset:", images.shape) #Size of that subset of dataset

images = images.astype("float32") / 255.0
masks = np.expand_dims((masks > 0).astype("float32"), -1) #Adding channel dimension and converting
    it to binary as required by cnn

imgtrain, imgtest, labeltrain, labeltest = train_test_split( #splitting data as training and testing
     data
    images, masks, test_size=0.1, random_state=42
)

print("Train images:", imgtrain.shape) #Getting their sizes
print("Test images:", imgtest.shape)

def dicecoefficient(truemask, predmask, smooth=1e-6):
    truemaskflat = K.flatten(truemask) #Converting them to 1d vectors
    predmaskflat = K.flatten(predmask)
    intersection = K.sum(truemaskflat * predmaskflat) #Getting their overlapped area
    return (2. * intersection + smooth) / (K.sum(truemaskflat) + K.sum(predmaskflat) + smooth)

def diceloss(truemask, predmask): #Loss function which we need to minimise
    return 1 - dicecoefficient(truemask, predmask)

def buildunet(input_shape=(56, 56, 1)): #Defining training function
    inputs = layers.Input(shape=input_shape) #Getting input

    # Encoder
    convo1 = layers.Conv2D(16, 3, activation='relu', padding='same')(inputs) #Applying first
        convolutional layer
    convo1 = layers.Conv2D(16, 3, activation='relu', padding='same')(convo1)
    pool1 = layers.MaxPooling2D()(convo1)

    convo2 = layers.Conv2D(32, 3, activation='relu', padding='same')(pool1) #Second convolutional
        layer
    convo2 = layers.Conv2D(32, 3, activation='relu', padding='same')(convo2)
    pool2 = layers.MaxPooling2D()(convo2)

    convo3 = layers.Conv2D(64, 3, activation='relu', padding='same')(pool2) #Third convolutional
        layer
    convo3 = layers.Conv2D(64, 3, activation='relu', padding='same')(convo3)
    pool3 = layers.MaxPooling2D()(convo3)

    #Bottleneck
    bot = layers.Conv2D(128, 3, activation='relu', padding='same')(pool3)#getting high level
        features
    bot = layers.Conv2D(128, 3, activation='relu', padding='same')(bot)

    # Decoder
    upsam3 = layers.UpSampling2D()(bot)  #Upsampling done to get back original size output
    upsam3 = layers.concatenate([upsam3, convo3])
    con4 = layers.Conv2D(64, 3, activation='relu', padding='same')(upsam3)
    con4 = layers.Conv2D(64, 3, activation='relu', padding='same')(con4)

    upsam2 = layers.UpSampling2D()(con4) #Upsampling done again
```

```
    upsam2 = layers.concatenate([upsam2, convo2])
    con5 = layers.Conv2D(32, 3, activation='relu', padding='same')(upsam2)
    con5 = layers.Conv2D(32, 3, activation='relu', padding='same')(con5)

    upsam1 = layers.UpSampling2D()(con5) #Last upsampling round
    upsam1 = layers.concatenate([upsam1, convo1])
    con6 = layers.Conv2D(16, 3, activation='relu', padding='same')(upsam1)
    con6 = layers.Conv2D(16, 3, activation='relu', padding='same')(con6)

    outputs = layers.Conv2D(1, 1, activation='sigmoid')(con6) #final output layer

    model = models.Model(inputs, outputs) #Linking input and output
    return model

model = buildunet() #Performing function and printing architechture in systematic manner
model.summary()

model.compile(optimizer=Adam(1e-4), #Difing optimizer , loss and metrice
              loss=dice_loss,
              metrics=[dice_coefficient])

history = model.fit(
    imgtrain, labeltrain,
    validation_data=(imgtest, labeltest),
    epochs=5,              # reduced for quick run
    batch_size=8,          # small batch to prevent session crashing
    verbose=1
)

testdice = model.evaluate(imgtest, labeltest, verbose=0)[1] #Getting dice coefficent
print(f"\n Test Dice Coefficient: {testdice:.4f}")

#Sample output results
n = 3
preds = model.predict(imgtest[:n])

#Binarize both ground truth and predictions for clarity
predsbin = (preds > 0.5).astype(np.uint8)
labeltestbin = (labeltest[:n] > 0.5).astype(np.uint8)

plt.figure(figsize=(12, 6))
for i in range(n):
    # Input
    plt.subplot(3, n, i + 1)
    plt.imshow(imgtest[i].squeeze(), cmap='gray')
    plt.title("Input")
    plt.axis('off')

    # Ground Truth
    plt.subplot(3, n, i + 1 + n)
    plt.imshow(labeltestbin[i].squeeze(), cmap='gray')
    plt.title("Ground Truth")
    plt.axis('off')

    # Prediction
    plt.subplot(3, n, i + 1 + 2 * n)
    plt.imshow(predsbin[i].squeeze(), cmap='gray')
    plt.title("Predicted")
    plt.axis('off')

plt.tight_layout() #Displaying output
plt.show()
```

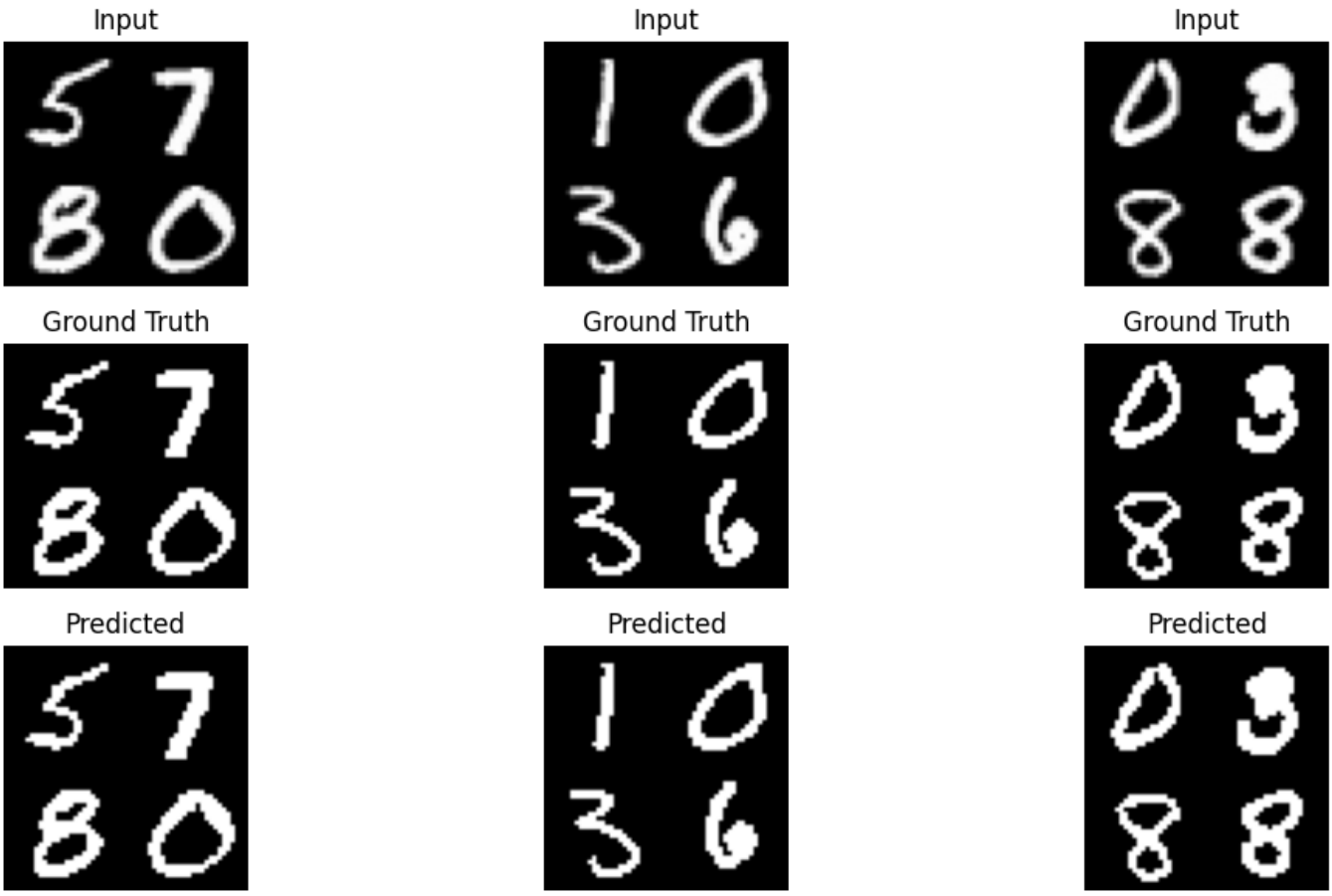

Figure 9: Sample output results

```
 Total params: 487,009 (1.86 MB)
 Trainable params: 487,009 (1.86 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
2250/2250 ━━━━━━━━━━━━━━━━━━━━ 658s 289ms/step - dice_coefficient: 0.8114 - loss: 0.1886 - val_dice_coefficient: 0.9657 - val_loss: 0.0343
Epoch 2/5
2250/2250 ━━━━━━━━━━━━━━━━━━━━ 668s 297ms/step - dice_coefficient: 0.9705 - loss: 0.0295 - val_dice_coefficient: 0.9801 - val_loss: 0.0199
Epoch 3/5
2250/2250 ━━━━━━━━━━━━━━━━━━━━ 658s 292ms/step - dice_coefficient: 0.9815 - loss: 0.0185 - val_dice_coefficient: 0.9870 - val_loss: 0.0130
Epoch 4/5
2250/2250 ━━━━━━━━━━━━━━━━━━━━ 669s 287ms/step - dice_coefficient: 0.9880 - loss: 0.0120 - val_dice_coefficient: 0.9917 - val_loss: 0.0083
Epoch 5/5
2250/2250 ━━━━━━━━━━━━━━━━━━━━ 674s 283ms/step - dice_coefficient: 0.9922 - loss: 0.0078 - val_dice_coefficient: 0.9940 - val_loss: 0.0060

 Test Dice Coefficient: 0.9940
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 462ms/step
```

Figure 10: Test Dice Coefficient Calculation

0.10 Dice Coefficient = 0.9940

The Dice Coefficient, also known as the Sørensen–Dice Index, measures the similarity between the predicted segmentation mask and the ground truth mask.

$$\text{Dice Coefficient} = \frac{2 \times |P \cap G|}{|P| + |G|}$$

Alternatively, it can be expressed in terms of pixel values as:

$$\text{Dice Coefficient} = \frac{2 \sum_i P_i G_i}{\sum_i P_i + \sum_i G_i}$$

where:

- P_i represents the predicted pixel value (1 for foreground, 0 for background),

- G_i represents the ground truth pixel value,
- $|P \cap G|$ denotes the number of pixels where both prediction and ground truth are 1.

0.11 Logic Explanation

U-Net Based CNN for Semantic Segmentation of MNIST Digit Grids
The U-Net based Convolutional Neural Network (CNN) is designed for pixel-wise segmentation of MNIST digit grid images using binary masks.

Dataset Preparation

- MNIST grid images and corresponding segmentation masks are loaded from pre-saved `.npy` files.
- A subset of 20,000 samples is used to optimize memory and training efficiency.
- Images are normalized to the range $[0, 1]$, and masks are binarized where 0 denotes background and 1 denotes foreground.
- The dataset is divided into 90% training and 10% testing sets using `train_test_split`.

Model Architecture – U-Net

The U-Net follows a symmetric encoder–decoder structure that captures both global context and fine spatial details.

- **Encoder (Contracting Path):** Three convolutional blocks with 16, 32, and 64 filters extract spatial and texture features, each followed by MaxPooling to down-sample dimensions.
- **Bottleneck Layer:** Two convolutional layers with 128 filters capture deep semantic features and contextual information.
- **Decoder (Expanding Path):** UpSampling layers restore the original spatial resolution. Skip connections concatenate encoder outputs to preserve boundary details.
- **Output Layer:** A 1×1 convolution with a sigmoid activation produces a binary segmentation mask representing foreground probability.

Training Setup

- The model is compiled using the `Adam` optimizer with a learning rate of 1×10^{-4}.
- **Dice Loss** is used as the loss function, and **Dice Coefficient** is used as the performance metric.
- Training is performed for 5 epochs with a batch size of 8 for stable convergence.

Evaluation

- Model performance is evaluated on the test set using the Dice Coefficient to measure segmentation accuracy.
- Predicted masks are thresholded at 0.5 for binary visualization.
- Sample input images, corresponding ground-truth masks, and predicted outputs are displayed for qualitative comparison.